

Academic Collaboration Network: Enhanced System Design and Project Management

Workshop #3 — Robust System Design and Project Management

Gabriel Andres Beltran Varela

Computer Engineering Program

Universidad Distrital Francisco José de Caldas

Bogotá, Colombia

gbeltranv@udistrital.edu.co

Kevin Santiago Silva Gonzalez

Computer Engineering Program

Universidad Distrital Francisco José de Caldas

Bogotá, Colombia

ksilvas@udistrital.edu.co

Miguel David Tarazona Correa

Computer Engineering Program

Universidad Distrital Francisco José de Caldas

Bogotá, Colombia

mtarazonac@udistrital.edu.co

Anyelo Esteban Casas Zapata

Computer Engineering Program

Universidad Distrital Francisco José de Caldas

Bogotá, Colombia

acasasz@udistrital.edu.co

Abstract—This document presents the enhanced system design and comprehensive project management plan for the Academic Collaboration Network, building upon Workshop #1 (systems analysis) and Workshop #2 (system design). This third workshop integrates robust engineering principles, quality assurance frameworks (ISO 9001, IEEE 730), risk management methodologies (PMBOK, ISO 31000), and a complete project management plan—including critical path analysis, Gantt scheduling, and capacity planning—to ensure successful implementation. Key enhancements include fault-tolerant microservices architecture, 10-risk PMBOK register with quantitative scoring, 99.5% uptime target, 80% code coverage goal, and 7-month delivery within \$50,000 USD budget. The document demonstrates clear design evolution from exploratory analysis through structural design to production-ready implementation planning.

I. EXECUTIVE SUMMARY

A. Document Overview

This document presents the enhanced system design and comprehensive project management plan for the Academic Collaboration Network, building upon Workshop #1 (systems analysis) and Workshop #2 (system design). This third workshop integrates robust engineering principles, quality assurance frameworks (ISO 9001, IEEE 730), risk management methodologies (PMBOK, ISO 31000), and a complete project management plan—including critical path analysis, Gantt scheduling, and capacity planning—to ensure successful implementation.

B. Design Evolution Summary

C. Expected Outcomes

- **Reliability:** 99.5% system uptime, < 2 s API response time (p95)
- **Quality:** $\geq 80\%$ code coverage, zero critical security vulnerabilities

- **Risk Mitigation:** 10 identified risks with specific, measurable mitigation strategies
- **Project Success:** 7-month delivery within \$50,000 USD budget
- **User Adoption:** 500+ concurrent users, 80% platform adoption rate
- **Usability:** SUS Score ≥ 70 , task completion rate $\geq 80\%$

II. ROBUST ARCHITECTURE DESIGN

A. Enhanced Architecture Overview

The enhanced architecture from Workshop #2 has been refined to incorporate fault-tolerance, horizontal scalability, observability, and API lifecycle management. The design follows the principles of the reactive manifesto (responsive, resilient, elastic, message-driven) applied to the microservices ecosystem. Each service is designed with clear boundaries, independent deployability, and specific failure modes addressed.

B. Fault-Tolerance Mechanisms

Fault tolerance is implemented at every layer of the stack, ensuring that no single component failure propagates into a system-wide outage. Table II details the failure modes and mitigation strategies by component, including Recovery Time Objectives (RTO) and Recovery Point Objectives (RPO) where applicable.

C. Scalability Enhancements

Horizontal scalability is achieved through Kubernetes-native autoscaling mechanisms and database connection pooling. Table III presents the scalability configuration by layer with specific thresholds for each mechanism.

Table I: Progressive Design Evolution Across Three Workshops

Dimension	Workshop #1	Workshop #2	Workshop #3
Focus	Problem identification	Solution design	Implementation readiness
Maturity	Exploratory	Structural	Operational
Key Output	Survey data (n=25)	Microservices blueprint	QA + Risk + PMP
Architecture	Conceptual context diagram	7 microservices	Fault-tolerant, multi-region
Quality	None defined	Basic testing plan	ISO 9001 + IEEE 730
Risk	Informal listing	5-risk critical matrix	10-risk PMBOK/ISO register
Management	None	Basic 7-month timeline	Full PMP: CPM, Gantt, budget

Table II: Fault-Tolerance Strategy by Component

Component	Failure Mode	Mitigation Strategy (RTO)
API Gateway	Service outage	Multi-region DNS failover + health checks (< 5 min)
PostgreSQL Primary	Primary DB failure	Automated replica promotion (Patroni) (< 5 min)
Redis Cache	Cache miss / node fail	Persistent backup + warm standby replica (< 2 min)
Matching Engine	Processing backlog	Async queue (RabbitMQ) + Dead Letter Queue (< 1 min)
Notification Service	Delivery failure	Exponential backoff (3 retries), idempotency (< 30 sec)
WebSocket Layer	Connection instability	Fallback to polling + circuit breaker pattern (< 10 sec)
Integration Gateway	LMS API unavailability	Cached last-known state + manual sync fallback (< 15 min)

Table III: Scalability Configuration by Layer

Layer	Mechanism	Threshold
Compute (HPA)	Kubernetes HPA; min 2 replicas/service	CPU > 70%
Database	PgBouncer pooling: 1,000 app → 100 DB connections	Conn > 80%
Cache	Redis Cluster sharding, 3 master nodes	Memory > 75%
CDN	CloudFront edge caching	Cache miss > 10%
Message Queue	RabbitMQ consumer groups with auto-scaling	Queue depth > 500

D. Maintainability Improvements

Maintainability is addressed through API versioning, structured logging, distributed tracing, and comprehensive health checks. Table IV details each feature and its implementation approach.

III. QUALITY ASSURANCE FRAMEWORK

A. Standards Compliance

The quality framework aligns with two primary international standards: ISO 9001:2015 for quality management systems and IEEE 730-2014 for software quality assurance processes.

1) *ISO 9001:2015 — Quality Management Systems*: Table V presents the implementation of relevant ISO 9001:2015 clauses in the context of software development for the Academic Collaboration Network.

2) *IEEE 730-2014 — Software Quality Assurance*: The IEEE 730 standard defines the following SQA Plan requirements applied to this project:

- **SQA Plan**: Maintained as a living document in the GitHub repository under /docs/SQA-Plan.md
- **Peer Reviews**: Mandatory for all design documents, pull requests, and architecture decisions
- **Testing Levels**: Unit → Integration → System → Acceptance (four-tier strategy)
- **Configuration Management**: Git-based SCM with branch protection rules and signed commits
- **Release Management**: Staged gates: development → staging → canary → production
- **Problem Reporting**: Jira issue tracking with severity classification and resolution SLAs

B. Quality Metrics and Targets

Table VI presents the complete quality metrics dashboard including targets, categories, and measurement tools for each metric.

Table IV: Maintainability Features and Implementation

Feature	Implementation
API Versioning	/v1/, /v2/ endpoints with 6-month deprecation policy and migration guides
Structured Logging	JSON format with correlation IDs, trace IDs, and severity levels (ELK stack)
Distributed Tracing	Jaeger integration for full request-lifecycle tracking across services
Health Checks	Liveness (/health/live) and readiness (/health/ready) Kubernetes probes
API Documentation	OpenAPI 3.0/Swagger for all public endpoints, auto-generated from code
Feature Flags	LaunchDarkly integration for zero-downtime feature rollouts
Dependency Scanning	Snyk + npm audit on every pull request for vulnerability detection

Table V: ISO 9001:2015 Clause Implementation

Clause	Implementation
7.1 Resources	Dedicated team structure, cloud infrastructure budget allocation
7.2 Competence	Training plan, skill assessments, certification requirements
7.5 Documentation	GitHub Wiki, OpenAPI specs, Architecture Decision Records (ADRs)
8.3 Design	Architecture Review Board, design documentation standards
8.4 External Providers	SLA agreements, API versioning, integration test suites
9.1 Monitoring	Metrics dashboard, PagerDuty alerting, weekly review meetings
9.2 Internal Audit	Bi-weekly peer code reviews, static analysis gates
10.1 Improvement	Bi-weekly retrospectives, user feedback loop, PDCA cycle

Table VI: Quality Metrics Dashboard

Metric	Target	Category	Tool
Code Coverage	$\geq 80\%$	Code Quality	Jest/Pytest + Codecov
API Response Time (p95)	< 500 ms	Performance	New Relic / Datadog APM
Matching Latency (p95)	< 3 s	Performance	App-level timer
System Uptime	99.5%	Reliability	Pingdom + AWS CloudWatch
Error Rate (5xx)	$< 1\%$	Reliability	APM monitoring
Cache Hit Rate	$\geq 90\%$	Performance	Redis INFO stats
SUS Usability Score	≥ 70	Usability	Post-launch survey
Task Completion Rate	$\geq 80\%$	Usability	UAT moderated sessions
Bug Resolution (Critical)	< 48 hours	Maintainability	Jira SLA tracking
Security Vulnerabilities	Zero critical	Security	Snyk + OWASP ZAP
NPS Score	≥ 40	Satisfaction	Post-test survey

C. Validation Approaches and Acceptance Criteria

Table VII presents the comprehensive testing strategy across all test levels, including scope, tools, and acceptance gates.

Table VIII presents the load testing scenarios with specific load levels and pass criteria for each scenario.

D. CI/CD Quality Gates

The following pipeline enforces quality gates before every production deployment:

- 1) Run unit tests (Jest / Pytest) — gate: all pass, coverage $\geq 80\%$
- 2) Upload coverage report to Codecov — gate: no regression
- 3) Snyk dependency vulnerability scan — gate: zero high/critical
- 4) SonarQube static analysis — gate: quality gate passed
- 5) Build and tag Docker images
- 6) Run integration tests (Supertest / Postman collections)
- 7) Deploy to staging environment
- 8) Run end-to-end tests (Cypress)
- 9) OWASP ZAP automated security scan — gate: zero

Table VII: Comprehensive Testing Strategy

Test Level	Scope & Approach	Tools	Gate
Unit	Individual functions and modules; mocked dependencies	Jest, Pytest	$\geq 80\%$ coverage
Integration	Service-to-service contracts; API endpoint validation	Supertest, Postman	All endpoints pass
End-to-End	Critical user journeys: registration, matching, workspace	Cypress, Selenium	$\geq 90\%$ paths pass
Performance	Load, stress, spike, endurance scenarios	JMeter, k6	Target met at 500 users
Security	OWASP Top 10; penetration testing	OWASP ZAP, Snyk	Zero critical vulns
Accessibility	WCAG 2.1 compliance AA	axe DevTools, WAVE	AA level achieved
UAT	30–50 beta users; guided onboarding; SUS & NPS surveys	Manual + surveys	SUS ≥ 70

Table VIII: Load Testing Scenarios

Scenario	Load	Pass Criteria
Baseline	50 users	< 500 ms p95
Peak	500 users	< 2 s p95, 0 errors
Stress	1,000 users	Graceful degradation
Spike	10 $\times$ surge (5 min)	Recovery < 2 min
Endurance	500 users / 24 h	No degradation

critical findings

10) If all gates pass: blue-green production deployment

IV. RISK MANAGEMENT PLAN

A. Risk Identification Framework

The risk management plan follows both PMBOK (7th Edition) and ISO 31000:2018 guidelines. Risks are categorized into four domains: Technical, Operational, Security, and Project Management. Risk scores are computed as $\text{Score} = P \times I$, where Probability (P) and Impact (I) are rated on a 1-5 scale.

B. Comprehensive Risk Register

Table IX presents the complete risk register with 10 identified risks, their probability and impact scores, and specific mitigation strategies for each.

C. Risk Mitigation Strategies

Each risk has a specific mitigation strategy as detailed below:

- **R1 (Algorithm bias):** A/B testing framework, manual fallback, bias audit monthly
- **R2 (LMS API failure):** Mock API for testing, fallback manual sync, SLA agreement
- **R3 (DB degradation):** Read replicas, PgBouncer pooling, query optimization sprint
- **R4 (WebSocket instability):** Polling fallback, circuit breaker, connection retry logic
- **R5 (Low adoption):** Gamification, faculty ambassadors, early-access program
- **R6 (Peak downtime):** Auto-scaling, multi-AZ deployment, pre-peak load tests

- **R7 (Security breach):** OAuth 2.0 + MFA, TLS 1.3, quarterly penetration testing
- **R8 (Schedule slippage):** 20% buffer time, daily standups, earned value tracking
- **R9 (Resource unavailability):** Cross-training, full documentation, backup assignments
- **R10 (Scope creep):** Change Control Board process, MVP backlog priority

D. Contingency Plans for Critical Scenarios

Table X presents contingency plans for critical failure scenarios with immediate response actions, Recovery Time Objectives (RTO), and Recovery Point Objectives (RPO).

V. PROJECT MANAGEMENT PLAN

A. Project Charter

Table XI presents the complete project charter summary including scope, objectives, success criteria, and constraints.

B. Team Structure and Roles

1) *Organizational Structure:* The team is organized with a Project Lead at the top, supported by four functional areas: Backend Development (2 developers), Frontend Development (2 developers), DevOps Engineering (1 engineer), and Quality Assurance (1 engineer), totaling 8 members.

2) *RACI Responsibility Matrix:* Table XII presents the RACI (Responsible, Accountable, Consulted, Informed) responsibility matrix for key project activities.

C. Project Timeline and Critical Path Analysis

1) *Milestone Schedule:* Table XIII presents the project milestone schedule with target dates and acceptance criteria for each milestone.

Table IX: Risk Register - PMBOK / ISO 31000 Framework

ID	Domain	Risk Description	P	I	Score
R1	Technical	Algorithm bias in matching engine	3	4	12
R2	Technical	LMS API integration failure	3	4	12
R3	Technical	Database performance degradation	2	4	8
R4	Technical	WebSocket connection instability	3	3	9
R5	Operational	Low user adoption	4	5	20
R6	Operational	Server downtime during exam peak	3	4	12
R7	Security	Data privacy breach	2	5	10
R8	Project Mgmt	Schedule slippage	3	4	12
R9	Project Mgmt	Key resource unavailability	2	3	6
R10	Project Mgmt	Scope creep	3	3	9

Table X: Contingency Plans for Critical Scenarios

Scenario	Immediate Response	RTO	RPO
Primary database failure	Automated failover to read replica (Patroni)	< 5 min	< 1 min
Complete region outage	DNS failover to secondary region (Route 53)	< 15 min	< 5 min
Security breach	Isolate systems, notify stakeholders, rotate credentials	< 1 h	N/A
Zero-day vulnerability	Emergency patch, activate WAF rules, restrict access	< 4 h	N/A
Production critical bug	Rollback via blue-green deployment switch	< 5 min	N/A
Low adoption (< 20%)	Activate faculty ambassador program, run workshops	2 weeks	N/A

Table XI: Project Charter Summary

Element	Description
Project Name	Academic Collaboration Network (ACN)
Project Sponsor	Universidad Distrital — Computer Engineering Program
Project Manager	Team 8 Lead (rotating per sprint)
Start Date	April 1, 2026
End Date	October 31, 2026 (7 months)
Total Budget	\$50,000 USD (estimated)
Objective	Deploy a scalable academic collaboration platform serving 500+ concurrent users
Success Criteria	99.5% uptime, < 2 s response time, 80% user adoption, SUS $\geq$ 70
Constraints	Fixed 7-month timeline, \$50,000 budget cap, 4-person core team
Assumptions	Cloud infrastructure accessible; LMS API credentials obtainable; students willing to pilot

Table XII: RACI Responsibility Matrix

Activity	Lead	Backend	Frontend	DevOps	QA
Requirements Definition	R/A	C	C	C	C
Architecture Design	A	R	C	C	C
Backend Development	I	R	C	C	C
Frontend Development	I	C	R	C	C
Code Review	A	R	R	C	C
Unit Testing	I	R	R	I	C
Integration Testing	I	C	C	C	R
CI/CD Pipeline	I	C	C	R	C
Security Audit	A	C	C	R	C
Production Deployment	A	C	C	R	C
Documentation	R	C	C	C	C
Stakeholder Reporting	R/A	I	I	I	I

R=Responsible, A=Accountable, C=Consulted, I=Informed

Table XIII: Project Milestone Schedule

ID	Target	Deliverable & Acceptance Criteria
M1	End Month 1	Architecture approved, cloud infrastructure provisioned, CI/CD operational
M2	End Month 2	Auth + Profile services deployed to staging; unit tests passing
M3	End Month 3	Internal beta (50 users); matching < 3 s; 90% satisfaction
M4	End Month 4	LMS + Library integration complete; 80% course sync success
M5	End Month 5	Mobile app available on TestFlight/Play Beta; 200 active users
M6	End Month 6	ML v2 deployed; p95 response < 500 ms; analytics dashboard live
M7	End Month 7	500+ concurrent users; 99.5% uptime; SUS $\geq$ 70

2) *Critical Path Analysis*: The critical path determines the minimum project duration. Activities on the critical path (float = 0) cannot be delayed without extending the project end date. Table XIV presents the critical path activity network.

Table XIV: Critical Path Activity Network

ID Predecessor	Activity	Duration	ES	EF
A1 —	Infrastructure Setup & CI/CD	3 w	W1	W3
A2 A1	Auth Service Development	3 w	W3	W6
A3 A1	User Profile Service	3 w	W4	W7
A4 A2	Skill Matching Engine (MVP)	4 w	W6	W10
A5 A3	Group Workspace Service	3 w	W7	W10
A6 A2	Notification Engine	2 w	W8	W10
A7 A4, A5	MVP Beta Deployment	1 w	W10	W11
A8 A7	LMS Integration Gateway	3 w	W11	W14
A9 A7	Library Integration	2 w	W11	W13
A10 A7	Mobile App Development	4 w	W11	W15
A11 A8	ML v2 (Enhanced Matching)	3 w	W14	W17
A12 A10	Analytics Dashboard	2 w	W15	W17
A13 A11	Performance Optimization	2 w	W17	W19
A14 A13	UAT & Final QA	2 w	W19	W21
A15 A14	Production Launch	1 w	W21	W22

Critical Path: A1→A2→A4→A7→A8→A11→A13→A14→A15

D. Resource Management and Capacity Planning

1) *Budget Allocation*: Table XV presents the detailed budget breakdown for the \$50,000 USD total budget.

Table XV: Budget Breakdown (\$50,000 USD Total)

Category	Allocation	Percentage
Cloud Infrastructure (AWS/GCP)	\$15,000	30%
Development (tools/contractors)	\$20,000	40%
Testing & QA	\$5,000	10%
Security Audit & Penetration Testing	\$5,000	10%
Training & Documentation	\$3,000	6%
Contingency Reserve	\$2,000	4%

2) *Infrastructure Capacity Planning*: Table XVI presents the infrastructure capacity planning across the three project phases.

3) *Human Resource Allocation*: Table XVII presents the team time allocation by phase as percentage of working time.

E. Communication and Control Plan

Table XVIII presents the stakeholder communication matrix including frequency, channel, and responsible party for each stakeholder group.

VI. IMPLEMENTATION STRATEGY AND READINESS

A. Phased Deployment Approach

1) *Phase 1 — MVP Foundation (Months 1-3)*: **Goal**: Validate core functionality with 50 beta users before scaling.

Deliverables: User authentication, profile creation, basic matching engine, group workspace, internal deployment.

Success Criteria: 50 beta users onboarded; matching latency < 3 s; satisfaction $\geq 90\%$; zero critical security vulnerabilities.

2) *Phase 2 — Integration & Enhancement (Months 4-5)*: **Goal**: Connect with institutional systems and release mobile application.

Deliverables: LMS (Moodle) OAuth integration, library resource search, React Native mobile app (iOS + Android beta).

Success Criteria: 200 active users; 80% course sync success; mobile app TestFlight/Play Beta live.

3) *Phase 3 — Optimization & Scale (Months 6-7)*: **Goal**: Production-ready system serving the full student cohort.

Deliverables: ML v2 matching, analytics dashboard, UX improvements, performance optimization, production launch.

Success Criteria: 500+ concurrent users; 99.5% uptime; API response < 2 s p95; SUS ≥ 70 .

B. Technical Infrastructure Requirements

Table XIX presents the infrastructure specifications and prerequisites for each component.

C. Training Plan

Table XX presents the training programs by audience including format, duration, and content.

D. Change Management Procedures

1) *Change Control Process*: The change control process follows these steps:

- 1) Change request submitted via Jira ticket (type: Change Request)
- 2) Impact assessment completed within 24 hours by responsible engineer
- 3) Change Control Board (CCB) review at weekly meeting
- 4) Approval / rejection communicated within 48 hours
- 5) If approved: implementation in next sprint
- 6) Post-implementation review within 1 week of deployment

2) *Rollback Procedures*: Table XXI presents rollback procedures by component including method and Recovery Time Objective (RTO).

VII. EVOLUTION AND CONTINUOUS IMPROVEMENT

A. Design Evolution from Workshop #1 to Workshop #3

Table XXII presents the complete documented design evolution across all three workshops.

Table XVI: Infrastructure Capacity Planning by Phase

Resource	Phase 1 (MVP)	Phase 2 (Integration)	Phase 3 (Scale)
Kubernetes Nodes	2 nodes (n2-std-2)	3 nodes (n2-std-4)	3–5 nodes (auto-scale)
PostgreSQL	1 primary, 1 replica	1 primary, 2 replicas	1 primary, 3 replicas + read pool
Redis	Standalone (6 GB)	Cluster (3 masters)	Cluster (3M+3R, 18 GB)
S3 Storage	20 GB	50 GB	100 GB + lifecycle policy
Concurrent Users	50 (beta)	200	500+
Monthly Cost (est.)	\$800/mo	\$1,400/mo	\$2,100/mo

Table XVII: Team Time Allocation by Phase

Role	Phase 1	Phase 2	Phase 3
Project Lead	40%	30%	30%
Backend Developer	90%	80%	60%
Frontend Developer	70%	80%	60%
DevOps Engineer	80%	50%	70%
QA Engineer	50%	60%	90%

B. Feedback Integration from Previous Workshops

The following improvements in Workshop #3 directly address gaps and reviewer feedback from Workshops #1 and #2:

- **Fault tolerance gaps (W#2):** Added per-component RTO/RPO, circuit breaker patterns, and multi-region DNS failover strategy
- **Informal risk management (W#1, W#2):** Formalized into 10-risk PMBOK register with quantitative scoring and contingency plans
- **Missing project schedule:** Completed with CPM analysis, Gantt chart, and milestone acceptance criteria
- **No capacity plan:** Added three-phase infrastructure sizing with monthly cost estimates
- **Adoption risk (R5, highest score):** Gamification, faculty ambassadors, and early-access program added as mitigations
- **Algorithm bias (R1):** A/B testing framework and manual fallback now explicitly designed into the matching engine

C. Continuous Improvement Framework — PDCA Cycle

The team applies the PDCA (Plan-Do-Check-Act) cycle throughout the project lifecycle:

- **Plan:** Define improvement objectives and success metrics based on retrospective findings
- **Do:** Implement changes in controlled environment (staging) with feature flags
- **Check:** Measure results against objectives using metrics dashboard and user feedback
- **Act:** Standardize successful changes or restart cycle with modified approach

D. Post-Launch Success Metrics

Table XXIII presents the post-launch evaluation framework with baseline and target metrics.

VIII. CONCLUSION

This document presented the enhanced system design and comprehensive project management plan for the Academic

Collaboration Network, demonstrating clear evolution from Workshop #1 (analysis) through Workshop #2 (design) to this robust, production-ready solution. Key achievements include:

- **Fault-tolerant microservices architecture** with per-component RTO/RPO specifications and multi-region failover
- **Quality assurance framework** aligned with ISO 9001:2015 and IEEE 730-2014 standards, including comprehensive testing strategy and CI/CD quality gates
- **Comprehensive risk management** with 10-risk PMBOK/ISO 31000 register, quantitative scoring, and specific mitigation strategies
- **Complete project management plan** including project charter, RACI matrix, critical path analysis, Gantt schedule, milestone tracking, and \$50,000 budget allocation
- **Phased implementation strategy** with three phases across 7 months, including training plans, change management, and rollback procedures
- **Continuous improvement framework** based on PDCA cycle with post-launch success metrics

The design achieves quantified delivery targets: 99.5% system uptime, < 2 s API response time, $\geq$ 80% code coverage, zero critical security vulnerabilities, 500+ concurrent users, and 80% platform adoption rate.

ACKNOWLEDGMENT

The authors thank Eng. Carlos Andrés Sierra, M.Sc., for guidance throughout the three workshops. We also acknowledge the 25 survey participants who contributed primary data to this project. Special thanks to the Computer Engineering Program at Universidad Distrital Francisco José de Caldas for supporting this research.

REFERENCES

- [1] ISO 9001:2015, *Quality Management Systems — Requirements*. International Organization for Standardization, 2015.
- [2] IEEE Std 730-2014, *IEEE Standard for Software Quality Assurance Processes*. IEEE Computer Society, 2014.
- [3] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 7th ed. Newtown Square, PA: PMI, 2021.
- [4] ISO 31000:2018, *Risk Management — Guidelines*. International Organization for Standardization, 2018.
- [5] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. O'Reilly Media, 2021.
- [6] M. T. Nygard, *Release It! Design and Deploy Production-Ready Software*, 2nd ed. Pragmatic Bookshelf, 2018.
- [7] D. W. Johnson, R. T. Johnson, and K. A. Smith, “Cooperative learning returns to college,” *Change*, vol. 30, no. 4, pp. 26-35, 1998.
- [8] S. Wasserman and K. Faust, *Social Network Analysis: Methods and Applications*. Cambridge University Press, 1994.

Table XVIII: Stakeholder Communication Matrix

Stakeholder	Frequency	Channel	Responsible
Team Members	Daily	Slack / Discord	All
Project Lead	Daily (standup)	Zoom / Meet (15 min)	Lead
Professor / Advisor	Bi-weekly	Email + Report	Lead
University Administration	Monthly	Presentation	Lead + Team
Beta Users (Students)	Bi-weekly	In-app + Email	Product Owner
External QA Firm	Quarterly	Formal report	Security Lead

Table XIX: Infrastructure Specifications and Prerequisites

Resource	Specification	Provider
Kubernetes Cluster	3-5 nodes, n2-standard-2/4, HPA enabled	GKE / EKS
PostgreSQL 14	1 primary + 1-3 replicas, Patroni HA	Cloud SQL / RDS
Redis 7	3 masters + 3 replicas, slot-based sharding	Memorystore / EC
S3-compatible Store	100 GB initial, lifecycle policies enabled	AWS S3 / GCS
CDN	Edge locations: US, EU, LATAM	CloudFront / GCP CDN
Monitoring Stack	Prometheus + Grafana + Pager-Duty alerts	Self-managed
CI/CD Pipeline	GitHub Actions + Docker Hub registry	GitHub
LMS API Credentials	Moodle REST API token (admin scope)	Universidad Distrital

Table XX: Training Programs by Audience

Audience	Format	Duration	Content
Students	Video tutorial + FAQ	10 min	Profile setup, skill matching, workspace usage
Professors	Live webinar	30 min	Student monitoring, group analytics access
Academic Advisors	Documentation + demo	1 hour	Isolation alerts, intervention workflows
Administrators	Workshop (hands-on)	2 hours	System configuration, backup, incident response

Table XXI: Rollback Procedures by Component

Component	Rollback Method	RTO
Backend Microservices	Blue-green deployment switch (K8s label swap)	< 5 min
Database Schema	Point-in-time recovery (PITR) via pg-restore	< 15 min
Frontend Web App	CDN cache invalidation + previous S3 artifact	< 2 min
Mobile App	Force-update via feature flag (LaunchDarkly)	< 1 hour
ML Matching Model	Model registry rollback to previous version	< 10 min

Table XXII: Documented Design Evolution Across Three Workshops

Aspect	Workshop #1	Workshop #2	Workshop #3
Understanding	Problem identification; survey (n=25) data	Solution design; microservices architecture	Robust implementation; operational readiness
Documentation	IEEE-format analysis paper	System design document	Management plan + QA + Risk
Architecture	Conceptual context diagram; AS-IS/TO-BE flows	7 microservices blueprint; DFD Level 0/1	Fault-tolerant; multi-region; observability
Quality	None defined	Basic testing strategy; 80% coverage target	ISO 9001 + IEEE 730; full QA pipeline
Risk	Informal listing in design doc	5-risk critical matrix	10-risk PMBOK/ISO 31000 register + CPM
Management	None	Basic 7-month timeline + 3 phases	Full PMP: charter, CPM, Gantt, RACI, budget, capacity
Key Finding	88% WhatsApp, 36% barrier, 96% intent ($\bar{x} = 3.88$)	FR/NFR traceability; matching algorithm formalized	Production readiness: contingency, rollback, training

- [9] K. S. Silva Gonzalez et al., “Workshop No. 1: Systems Analysis — Academic Collaboration Network,” Systems Analysis & Design, Universidad Distrital Francisco José de Caldas, Mar. 2026 (unpublished).
- [10] K. S. Silva Gonzalez et al., “Workshop No. 2: System Design — Academic Collaboration Network,” Systems Analysis & Design, Universidad Distrital Francisco José de Caldas, Mar. 2026 (unpublished).